Choosing the right model

Choosing a Claude model means balancing capabilities, speed, and cost. This guide covers the questions to ask, two ways to pick a starting model, and how to test the choice.

Establish key criteria

When choosing a Claude model, consider first evaluating these factors:

- **Capabilities:** What specific features or capabilities will you need the model to have to meet your needs?
- **Speed:** How quickly does the model need to respond in your application? Fast mode delivers up to 2.5x higher output speed at premium pricing on supported models.
- **Cost:** What's your budget for both development and production usage?
- **Effort:** Several Claude models support an effort parameter that trades intelligence for latency and cost within a single model. Tuning effort is often a better lever than switching models. On Claude Fable 5.2, Claude Fable 5.1, Claude Opus 5, and Claude Sonnet 5.5, start with the default (`high`) and adjust up or down based on your evals. On Claude Opus 5.5 and Claude Haiku 5.5, the default is `medium`; start there and adjust the same way. On Claude Opus 4.8 and Claude Opus 4.7, the `xhigh` effort level, between `high` and `max`, is the best setting for most coding and agentic use cases.

Choose the best model to start with

There are two general approaches you can use to start testing which Claude model best works for your needs.

Option 1: Start efficiency-first

For many applications, starting with a faster, more cost-effective model such as Claude Haiku 5.5 can be the optimal approach:

1. Start implementation with Claude Haiku 5.5.
2. Test your use case thoroughly.
3. Evaluate if performance meets your requirements.
4. Upgrade only if necessary for specific capability gaps.

This approach allows for quick iteration and lower development costs, and is often sufficient for many common applications. This approach is best for:

- Initial prototyping and development
- Applications with tight latency requirements
- Cost-sensitive implementations
- High-volume, straightforward tasks

Option 2: Start capability-first

For complex tasks where intelligence and advanced capabilities are paramount, you might want to start capability-first: implement with the strongest starting point for your task, then optimize to more efficient models later:

1. Implement with Claude Opus 5.5.
2. Optimize your prompts for this model.
3. Evaluate if performance meets your requirements.
4. Consider increasing efficiency by lowering effort or downgrading models over time with greater workflow optimization.
5. If your evals at `xhigh` or `max` effort still fall short on demanding reasoning or long-horizon agentic work, move to Claude Fable 5.2.

This approach is best for:

- Complex reasoning tasks
- Scientific or mathematical applications
- Tasks requiring nuanced understanding
- Applications where accuracy outweighs cost considerations
- Advanced coding and high-autonomy agentic work

Claude Opus 5.5 (`claude-opus-5-5`) is built for long-running agentic coding and knowledge work, with adaptive thinking always on. If your integration forces tool use (`tool_choice` of

type `any` or `tool`), turns thinking off, or uses the `computer_20251124` computer use tool, see Breaking changes before switching.

Claude Fable 5.2 (`claude-fable-5-2`) is Anthropic's most capable model open to all customers. It's built for demanding reasoning and long-horizon agentic work. **Claude Mythos 5.2** (`claude-mythos-5-2`) offers the same capabilities only to organizations verified through Anthropic's verification programs, such as the Cyber Verification Program. Both models use always-on adaptive thinking. If your code changes `system`, `tools`, or earlier messages in a conversation, moves a conversation between models, uses the `computer_20251124` computer use tool, or uses the advisor tool, see Claude Fable 5.2 breaking changes before switching.

For context windows, output limits, and prices, see the All models table.

Model selection matrix

Most workloads start with Claude Opus 5.5.

When you need...	Consider starting with...	Example use cases
The highest available capability	Claude Fable 5.2	Agent sessions that run for hours, multistep deep research, analysis carried through to a finished document, spreadsheet, or deck
Complex agentic coding and enterprise work	Claude Opus 5.5	Multihour autonomous coding agents, large-scale refactoring, complex systems engineering, vision-heavy workflows, computer use
Speed and capability for everyday coding, agent, and enterprise workloads	Claude Sonnet 5.5	Code generation, data analysis, content creation, visual understanding, agentic tool use
The lowest latency and price	Claude Haiku 5.5	Real-time applications, high-volume intelligent processing, cost-sensitive deployments needing strong reasoning, subagent tasks

Decide whether to upgrade or change models

To decide whether to upgrade or change models:

1. Create benchmark tests specific to your use case. Having a good evaluation set is the most important step in the process.
2. Test with your actual prompts and data.
3. Compare performance across models for:
    - Accuracy of responses
    - Response quality
    - Handling of edge cases
4. Weigh performance and cost tradeoffs.

Combine models

Multimodel strategies pair a lower-cost model with a frontier model so that most tokens are billed at the lower rate. The two common patterns are an executor that escalates hard decisions to an advisor, and an orchestrator that delegates bulk work to lower-cost workers. See Optimizing for cost and intelligence for both strategies, measured examples, and implementation options.

Next steps



Model comparison chart

See detailed specifications and pricing for the latest Claude models



What's new in Claude Fable 5.2

The latest Fable model: breaking changes, feature support, and pricing



What's new in Claude Opus 5.5

The latest Opus model: breaking changes, new features, and behavior differences



What's new in Claude Sonnet 5.5

The latest Sonnet model: breaking changes, new features, and behavior differences

What's new in Claude Haiku 5.5

The latest Haiku model: breaking changes, new features, and behavior differences

Start building

Get started with your first API call